

최종발표 15분 대본

진행 방식 (7기 OT p.7 원문 기준)

항목	내용
일정	최종발표 및 AI 면접 9/18 오후 2시 ~ 9/22 자정. 편한 시간에 자유롭게 응시
구성	최종발표 15분 + 질의응답 5문항, 5분씩
자료	발표자료 PDF · 시연영상 · 소스코드. 문항은 최종산출물 기반 자동 생성
주의	모의면접은 제공되지만 본 면접은 재응시 불가
채점	AI 자동채점 후 휴먼 심사위원 Cross-Check (Working Day 5일 소요)

PPTX 발표자 노트의 "10분 제한" 표기는 3기 발표템플릿 가이드 기준이다. 7기 OT는 15분이므로 이 대본을 따른다. 노트 분량은 실측 3,547자로 8.4~10.4분이라 그대로 읽으면 4~6분이 빈다. 아래에서 [보강] 표시가 그 빈 시간을 채우는 부분이고, 전부 제출 문서에 있는 내용이다.

15분 시간표

시각	슬라이드	내용	평가 기준
0:00~0:40	1 표지	인사 · 한 문장 소개 · 오늘 순서	—
0:40~1:40	2	문제 정의 — 무엇이 어려운가	1-1
1:40~2:40	2	[보강] 목표를 어떻게 구체화했나 · KPI 설정 방식	1-2
2:40~3:20	2	[보강] 8주 실행 계획과 우선순위	1-3
3:20~5:00	2	다섯 역할이 도는 방식 · 검증 수치와 분모	2-2
5:00~6:30	3	세 갈래 경로 · Task Context RAG	2-1
6:30~7:30	3	네 용어 구분 · Vector DB와 LangGraph를 안 쓴 이유	2-1
7:30~8:40	3	Python Guard · [보강] 외부 API와 DB 연계	2-2
8:40~10:00	4	난제 — 단일 결론은 검증할 수 없다	2-1
10:00~11:20	4	A/B로 켜 전후 차이	2-3
11:20~12:30	4	실패 두 건과 해결	3-3
12:30~13:20	4	[보강] 테스트를 어떻게 했나 (4겹)	2-3
13:20~14:00	4	남은 한계 · [보강] 확장하려면 무엇이 필요한가	3-4
14:00~14:20	4	마무리 한 문장	—

목표는 14:20이다. 40초를 여유로 남긴다.

아래 대사 분량은 실측 4,862자다. 분당 340자로 또박또박 읽으면 14.3분, 분당 420자로 빠르게 읽으면 11.6분이다. 즉 **천천히 읽어야 15분에 맞는다**. 빨라져서 12분대에 끝날 것 같으면 아래 **시간 조절**의 늘리는 항목을 쓴다. 15분을 넘기는 것보다는 낫지만, 3분을 비우는 것도 손해다.

0:00~0:40 · 슬라이드 1 (표지)

안녕하십니까. AI Master 7기 백준현입니다. 제가 만든 것은 **MailTaskAgent**, 메일로 들어오는 업무 요청을 업무 항목으로 만들고 뒤따라오는 메일에 맞춰 기한과 상태를 이어서 관리하는 개인 업무관리 에이전트입니다.

오늘은 네 가지를 말씀드리겠습니다. **첫째** 어떤 문제에서 시작했고 무엇을 목표로 잡았는지, **둘째** 그 목표를 어떤 구조로 풀었는지, **셋째** 가장 오래 막혔던 기술 과제와 그것을 어떻게 검증했는지, **넷째** 남은 한계와 앞으로의 방향입니다.

시계를 여기서 한 번 본다. 0:40을 넘겼으면 다음 구간에서 20초를 줄인다.

0:40~1:40 · 슬라이드 2 — 문제 정의

문제부터 말씀드리겠습니다. 하나의 업무가 여러 메일과 여러 Thread에 흩어져 도착하고, 기한 변경이나 자료 회신 같은 후속 메일은 표현이 매번 달라집니다. 그래서 사용자는 메일이 올 때마다 과거 메일을 다시 찾아, 이게 새 업무인지 기존 업무의 변경인지 직접 판단해야 합니다. 놓치면 기한을 넘기고, 잘못 붙이면 엉뚱한 업무의 상태가 바뀝니다. **제가 풀려는 것은 이 판단입니다**.

기존 방식으로는 두 가지가 있었습니다. 사람이 기억하거나 메모하는 방식은 메일 확인 습관에 의존하기 때문에 장기 대기 중인 업무가 눈에 띄지 않습니다. 메일 규칙이나 필터는 "부탁", "월요일" 같은 단어는 잡아냅니다. 그런데 그것이 **새 요청인지 기존 업무의 변경인지, 어느 업무에 붙는지는** 단어만으로 알 수 없습니다. 메일 문맥과 현재 업무 상태를 함께 봐야 하는데, 표현이 매번 다르므로 규칙으로 전부 열거할 수 없습니다.

1:40~2:40 · 슬라이드 2 — 목표 설정 [보강]

그래서 목표를 세울 때 한 가지를 먼저 정했습니다. **"메일 관리를 편하게 한다"**로 두면 측정할 수 없다는 것입니다. 그래서 정답을 미리 적을 수 있는 형태까지 목표를 내렸습니다.

핵심 시나리오 세 개를 대표 업무 케이스 열다섯 개로 쪼개고, **각 케이스의 기대 행동과 상태, 필수 필드를 구현 전에 확정했습니다**. 정답셋을 먼저 쓰고 구현을 나중에 했기 때문에, 나중에 결과가 안 나온다고 기준을 느슨하게 바꿔 통과시키는 일을 스스로 못 하게 만들었습니다.

지표는 업무 흐름을 분류, 필드 추출, 업무 연결, 행동 결정, 전체 반영 다섯 단계로 나누고 단계마다 하나씩 붙였습니다. 목표선은 분류와 추출과 전체 성공률이 90퍼센트, 업무 연결과 행동 일치율이 85퍼센트입니다. **뒤 둘을 낮게 잡은 이유**는 정답이 하나로 확정되지 않는 모호한 케이스가 섞이기 때문이고, 실제로 업무 연결은 정답이 하나로 확정되는 여덟 개 단계만 분모에 넣었습니다.

2:40~3:20 · 슬라이드 2 — 실행 계획 [보강]

8주 계획은 우선순위를 이렇게 뒀습니다.

첫째, 합성 데이터로 끝까지 한 바퀴 도는 것을 최우선으로 했습니다. 사내 Outlook 권한이 나오지 않아도 프로젝트가 완주될 수 있어야 했기 때문에, 실제 메일 연동을 필수 경로에서 뺐습니다.

둘째, 판단 품질보다 틀렸을 때 막는 구조를 먼저 만들었습니다. 일곱 개 행동과 다섯 개 상태, 그리고 파이썬 가드입니다. 업무 데이터는 되돌리기 어렵기 때문입니다.

셋째로 평가 스위트와 정답셋을 만들었습니다. 이게 있어야 이후 변경이 개선인지 후퇴인지 알 수 있습니다. 그 다음에 지메일 어댑터, 컨텍스트 검색과 숙고 경로, 마지막으로 가장 위험한 회신 발송을 붙였습니다. **가장 위험한 기능을 승인 게이트가 자리잡은 뒤에 붙인 것이 순서의 핵심입니다.**

3:20~5:00 · 슬라이드 2 — 구조와 검증 수치

가운데 다섯 상자는 실제로 도는 역할입니다. 메일 분석기가 의미와 의도를 구조화하고, 컨텍스트 에이전트가 관계와 대상, 행동 후보를 두세 개 만듭니다. **같은 에이전트를 한 번 더 부르는데 이번엔 평가만 시킵니다. 만드는 호출과 고르는 호출이 다릅니다.** 그다음 파이썬 가드가 후보와 상태 전이, 중요 변경을 검증하고 기준에 못 미치면 사용자 확인으로 넘깁니다. 마지막이 회신 에이전트입니다. **엘엘엠이 판단하는 자리와 파이썬이 통제하는 자리가 이 그림에서 같립니다.**

아래 수치는 **분모가 서로 다르니 합쳐 읽지 말아 주십시오.** 28 대 28은 회사 엘엘엠 라이브 열다섯 개 케이스에서 나온 행동 단계이고, 3 대 3은 컨텍스트 에이전트 전용 라이브, 20 대 20은 테스트 지메일의 비식별 합성 메일 수용시험, 파이테스트 228은 코드 회귀입니다. **회사 메일함 전체 정확도가 아닙니다.**

체감 시간은 신뢰할 기준선을 얻지 못해 추정치 대신 **미측정**으로 적었습니다. 측정하려면 같은 메일 묶음을 손으로 분류할 때와 이 도구를 쓸 때의 소요를 같은 사용자가 번갈아 재야 하는데, 사용자가 저 한 명이라 순서 효과를 걷어낼 방법이 없었습니다.

5:00~6:30 · 슬라이드 3 — 세 갈래 경로와 RAG

가장 중요한 설계 결정부터 말씀드리겠습니다. **모든 메일을 에이전트에게 맡기지 않았습니다.** 경로가 셋으로 갈립니다. 같은 Thread에 활성 업무가 정확히 하나면 규칙이 바로 연결하고 검색과 에이전트 판단을 아예 호출하지 않습니다. 관련 업무를 찾아야 할 때만 검색 경로로 가고, 업무 요청이 아니면 기존 규칙 경로로 처리합니다. **입력에 따라 호출 자체가 달라지는 것이 고정 워크플로우와의 차이입니다.**

엠 영이가 이 프로젝트의 알에이지입니다. **외부 문서를 찾는 지식 알에이지가 아니라, 판단에 필요한 과거 업무 맥락을 제 에스큐엘라이트에서 찾아오는 태스크 컨텍스트 알에이지입니다.** 활성 업무 다섯 건, 각 업무의 최근 메일 세 건과 변경 이력 다섯 건, 사용자가 과거에 확정한 결정을 함께 가져옵니다. 지난 결정이 다음 판단의 근거로 다시 들어갑니다.

6:30~7:30 · 슬라이드 3 — 용어 구분과 기술 선택

네 용어의 역할이 다릅니다. 알에이지는 맥락을 가져오는 단계, 리액트는 그 맥락을 관찰해 행동을 정하고 실행 결과를 다시 관찰하는 바깥 루프, 셀프 커렉션은 그 루프 안에서 확신이 부족할 때 검색어를 바꿔 한 번 다시 찾고 다시 판단하는 부분입니다. 파이썬 가드는 에이전트의 판단을 대신하는 자리가 아니라 **그 판단을 실행해도 되는지 확인하는 경계**입니다. 흐름으로 읽으면 관찰, 검색, 판단, 실행, 재관찰입니다.

벡터 디비를 쓰지 않은 이유는 개인의 활성 업무가 수십 건 규모라 에스큐엘라이트 조회만으로 충분했고, 구성 요소를 늘리는 비용이 더 컸기 때문입니다. 대신 검색이 어휘 겹침에 의존한다는 한계가 남습니다. 모델 간 비교 실험은 하지 않았으므로 최적이라고 주장하지 않습니다.

랭그래프도 쓰지 않았습니다. 에이전트가 하나이고 상태가 업무 다섯 개와 행동 일곱 개로 닫혀 있어, 파이썬 상태와 가드만으로 검증이 더 쉬웠습니다. 분기와 중단, 재개가 복잡해지는 시점에 다시 검토하겠습니다.

7:30~8:40 · 슬라이드 3 — Guard와 연계 [일부 보강]

세 번째가 가장 오래 고민한 **파이썬 가드**입니다. 엘엘엠은 제안만 하고 데이터베이스를 직접 바꾸지 않습니다. 가드는 고른 업무 아이디어가 검색 후보 안에 있는지, 메일 의도와 행동이 맞는지, 생성에 필요한 값이 있는지를 확인하고, 하나라도 어긋나면 사용자 확인으로 올립니다. **완료와 취소, 기한 단축은 신뢰도와 무관하게 항상 승인을 거칩니다.**

[보강] 외부 연계도 짧게 말씀드리겠습니다. **회사 엘엘엠**은 오픈에이아이 에스디케이 호환 클라이언트로 붙었고 응답은 파이단틱 스키마로 검증합니다. 스키마가 어긋나면 한 번 재시도하고, 최종 실패하면 **데이터베이스를 건드리지 않습니다. 지메일**은 읽기 전용 어댑터로 붙였고, 읽기 토큰과 발송 토큰을 분리했습니다. 발송은 원본 스레드, 허용 수신자 목록, 사용자 승인 **세 가지를 모두 만족해야** 실행됩니다. **에스큐엘라이트**는 하나의 트랜잭션으로 저장하고 저장 후 다시 조회해 실제 상태를 확인합니다. 화면과 1분 주기 수집기가 동시에 쓰기 때문에 WAL과 30초 대기, 단일 실행 잠금을 걸었습니다.

8:40~10:00 · 슬라이드 4 — 난제

이 슬라이드의 난제부터 말씀드리겠습니다. 저는 이미 에이전트를 쓰고 있었는데, **그 에이전트가 결론을 하나만 돌려줬습니다.** 관계 하나, 대상 하나, 행동 하나에 설명 한 줄입니다. 이러면 모델이 후보를 실제로 건졌는지, 결론을 먼저 정하고 설명을 붙였는지 밖에서 구분할 방법이 없습니다. 멘토님 피드백도 매번 같았습니다. **를 베이**스처럼 보이나, 어떤 후보를 검토했는지 보여 달라. **막힌 곳은 판단의 품질이 아니라 검증 가능성이었습니다.**

그래서 적용한 기법이 화면에 적힌 **제한적 다중 가설 숙고**입니다. 앞 장과의 관계를 먼저 정리하겠습니다. 숙고는 한 번의 판단 안에서 후보를 건주는 안쪽 단계이고, 리액트는 그 판단을 검색과 실행, 재관찰로 감싸는 바깥 루프입니다. **숙고가 리액트를 대체한 것이 아니라 리액트의 판단 단계를 두 호출로 쪼갠 것입니다.** 트리 오브 소트를 구현한 것이 아니라, 후보 생성과 평가를 분리한다는 아이디어만 참고했습니다.

메일 한 건을 업무로 옮기는 판단은 단계가 깊지 않은 대신 틀렸을 때 비용이 업무 데이터 오변경이라, 후보 두 세 개, 평가 한 번, 재검색 한 번으로 묶고 결정론적 가드와 사용자 승인으로 막는 쪽을 택했습니다. 핵심은 생성과 평가를 **서로 다른 호출로 나눈 것**입니다. 생성 단계는 점수를 매기지 않고, 평가 단계는 새로 만들지 못합니다. 그래서 화면에 남는 후보 목록이 **사후 설명이 아니라 실제로 비교된 기록**이 됩니다.

10:00~11:20 · 슬라이드 4 — 전후 차이

효과는 **같은 코드에서 기능만 켜다 켜서** 측정했습니다. 기존 열다섯 개 케이스는 양쪽이 똑같이 15 대 15, 행동 28 대 28이고 소요시간만 약 30퍼센트 늘었습니다. 숙고가 실제 발동하는 전용 네 개 케이스는 회차마다 출력이 달라져 각 설정을 세 번씩 돌렸고, 통과 수는 컨 쪽이 3, 3, 2, 끈 쪽이 2, 2, 2입니다.

세 번 모두 재현되는 차이는 한 건입니다. 요청자가 같고 대상 시스템만 다른 유사 업무가 둘 있을 때, 단일 결론 경로는 세 번 모두 둘 중 하나에 그대로 연결했고, 두 단계 경로는 세 번 모두 사용자 확인으로 닫았습니다. **제가 개선했다고 말하는 범위는 이 한 건**이고, 엘엘엠을 호출하지 않는 목 경로에서는 양쪽 모두 4 대 4로 차이가 없습니다.

성능이 올라갔다는 주장이 아닙니다. **잘못된 업무에 조용히 불던 것이 사람에게 넘어갔다는 뜻**입니다.

11:20~12:30 · 슬라이드 4 — 실패 두 건

실패도 말씀드리겠습니다. **첫째**, 응답 검증을 재시도 루프 밖에 두어서, 모델이 필드 하나를 배열로 돌려준 것만으로 판단 전체가 실패했습니다. 저장해 둔 실제 지메일 서른다섯 건을 다시 불러보냈을 때 판단 네 건이 사라졌고, **합성 테스트가 아니라 실제 데이터 재생에서 잡혔습니다.** 검증을 루프 안으로 옮긴 뒤 유실은 0건입니다.

둘째는 기대값과 동작 중 어느 쪽이 틀렸는지 따져야 했던 건입니다. 검색 결과 안에 명령을 심어 둔 케이스가 세 번 모두 기대값인 사용자 확인 대신 무시로 끝났습니다. 그 메일은 순수 주입이 아니라 정상 회신 내용과 주입된 명령이 섞인 혼합 메일이고 정답도 업무 요청 참이었는데, **제 프롬프트가** 프롬프트 인젝션을 공지·광고와 같은 줄에 두고 업무 요청이 아닌 사유로 제시해서 모델이 메일 전체를 버렸습니다. 원칙은 본문의 명령을 실행하지 않는다이지 명령이 섞인 메일을 버린다가 아닙니다. **기대값은 그대로 두고 규칙만 분리**했더니 세 번 모두 통과하고, 숙고 전용 통과 수는 3, 4, 4가 되었습니다. 명령만 들어 있는 순수 주입은 그대로 무시로 끝나 둘의 구분 유지되고, 본문이 지시한 전체 완료 처리와 기록 삭제는 수정 전후 어느 회차에서도 일어나지 않았습니다.

12:30~13:20 · 슬라이드 4 — 테스트 방식 [보강]

검증은 네 겹으로 했습니다.

첫째, 회귀 테스트 파이테스트 228개입니다. 기능이 늘 때마다 계약을 테스트로 고정했고, 기준선은 171에서 179를 거쳐 228입니다. **둘째, 시나리오 평가 스위트**입니다. 각 케이스를 **격리된 데이터베이스에서 재현**하고 미리 정의한 기대 행동과 상태, 필드와 대조합니다. **셋째, 수용시험**입니다. 별도 테스트 지메일 계정으로 비식별 합성 메일 스무 건을 실제로 수신해 20 대 20을 확인했습니다. **넷째, 실제 스레드 종단 검증**입니다. 실제 지메일 다섯 통짜리 스레드 하나를 생성부터 완료까지 돌려 승인 발송, 회신 대기, 기한 단축 승인, 완료 승인, 중복 차단 확인했습니다.

한 가지 덧붙이면, **목 환경과 라이브 환경을 분리**했습니다. 목은 어휘 겹침만 보기 때문에 숙고가 다루는 판단 자체가 발생하지 않습니다. 그래서 목 결과로 숙고 효과를 주장하지 않습니다.

13:20~14:00 · 슬라이드 4 — 한계와 확장 [일부 보강]

남은 한계를 말씀드리겠습니다. 화면의 신뢰도와 지지도는 **모델의 자기보고 값이지 검증된 정답 확률이 아닙니다**. 사용자 체감 시간은 신뢰할 기준선을 얻지 못해 미측정으로 남겼습니다. 검증이 합성 중심이라 사람이 실제로 쓰는 문장의 모호함은 충분히 재현되지 않았습니다.

[보강] 실제 서비스로 확장하려면 다섯 가지가 더 필요합니다. **입력 경계**는 마이크로소프트 그래프 등록과 사내 커넥터인데, 메일 어댑터만 교체하고 **에이전트 코어와 가드는 그대로 두는** 구조로 이미 설계했습니다. **인증과 권한**은 사내 에스에스오와 발송 권한 승인 체계, **저장소**는 다중 사용자 동시 쓰기를 감당할 운영 데이터베이스입니다. 에스큐엘라이트로는 안 됩니다. 나머지 둘은 운영 인프라와 보안정책 검토입니다.

가장 먼저 할 일은 **파일럿 사용자 서너 명으로 시간 기준선을 실제로 재는** 것입니다. 이번에 못 켜 그 지표입니다.

14:00~14:20 · 마무리

정리하겠습니다. 이 프로젝트가 한 일은 메일 판단을 자동화한 것이 아니라, **판단을 자동화하되 틀렸을 때 조용히 지나가지 않게 만든** 것입니다. 규칙으로 되는 것은 규칙이 하고, 안 되는 것만 에이전트가 판단하며, 실행 권한은 파이썬이 갖습니다. 확신이 없으면 사람에게 올립니다.

수치는 모두 정의된 범위 안의 값이고, 못 켜 것은 못 켜다고 적었습니다. 감사합니다.

시간 조절

말이 빨라져 시간이 남을 때 (12분대에 끝날 것 같으면)

- 3:20 구간에서 다섯 역할을 하나씩 더 풀어 설명한다
- 12:30 테스트 네 겹에서 각 층의 숫자를 한 번 더 짚는다

- 마무리에서 "질의응답에서 더 깊게 말씀드리겠습니다"를 붙인다

시간이 모자랄 때 (10:00에 아직 슬라이드 3이면)

- **[보강]** 표시 구간을 순서대로 줄인다: 2:40 실행 계획 → 12:30 테스트 → 7:30 외부 연계
- **절대 줄이면 안 되는 곳:** 8:40 난제, 10:00 전후 차이, 13:20 한계. 이 셋이 평가 핵심이다

발표 직전 점검

- 슬라이드 숫자를 기억으로 바꿔 말하지 않는다 — **15/15 , 28/28 , 20/20 , 8/15(53.3%)**
- 용어를 줄여 말하지 않는다 — "ToT", "ReAct Agent"가 아니라 슬라이드에 적힌 그대로
- "정확도 100%" 뒤에 **"정의된 합성 15 Case에서"**를 반드시 붙인다
- 주 1.2~2.8시간은 **실측이 아니라 외부 통계 기반 잠재치**라고 말한다
- 한 번은 실제로 소리 내어 읽고 시간을 잰다